

Git, Github e Ambientes de Desenvolvimento

Eduardo Adame

Ciência de Dados

11 de março de 2026



Por que controle de versão?



Problemas comuns em projetos:

- Arquivos chamados:
 - ▶ analysis_final.py
 - ▶ analysis_final_v2.py
 - ▶ analysis_final_definitivo.py
- Mudanças irreversíveis
- Colaboração difícil
- Código quebrando sem saber quando

Objetivo do Git

Rastrear mudanças em arquivos ao longo do tempo.

Snapshots, não diffs



Git armazena o estado do projeto como **snapshots**.



histórico de commits

Cada commit contém:

- snapshot dos arquivos
- autor
- mensagem
- hash SHA



Componentes principais:

- Working Directory
- Staging Area
- Repository





Listing 1: Inicializando repositório

```
1 git init
2 git add file.py
3 git commit -m "first commit"
```



Listing 2: Clonar repositório

```
1 git clone git@github.com:adamesalles/ciencia-de-dados.git
```

Listing 3: Enviar alterações

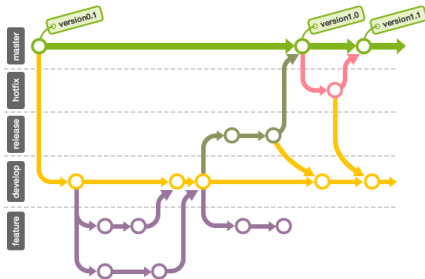
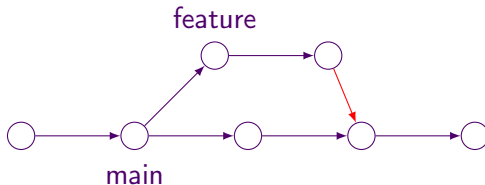
```
1 git push origin main
```

Listing 4: Atualizar repositório

```
1 git pull
```

Branching

Branches permitem desenvolver funcionalidades isoladamente.





Listing 5: Branch

```
1 git branch feature
2 git checkout feature
3
4 # ou
5
6 git checkout -b feature
```

Tipos de merge



Tipos comuns:

- Fast-forward
- Merge commit
- Rebase

Fast-forward ocorre quando não há divergência no histórico.



Corrigir último commit:

Listing 6: Amend

```
1 git commit --amend  
2 git push --force-with-lease # cuidado!!
```

Usos comuns:

- corrigir mensagem
- adicionar arquivo esquecido



Issues são usadas para organizar tarefas, bugs e novas funcionalidades.

- Cada issue representa um **problema ou tarefa**
- Possui número único: #12
- Pode ter responsáveis, labels e discussões

Pull Requests (PR) são propostas de mudança no código.

- Um branch é enviado para revisão
- Outros desenvolvedores podem comentar e revisar
- Após aprovação, o código é integrado ao branch principal



Podemos mencionar issues usando #numero.

Listing 7: Referência em commit

```
1 git commit -m "implementa leitura de palavras (#12)"
```

Algumas palavras-chave fecham a issue quando o commit/PR é integrado ao branch principal:

Listing 8: Fechando issue automaticamente

```
1 git commit -m "implementa lógica do jogo. closes #12"
```

Palavras aceitas: close, closes, closed, fix, fixes, fixed, resolve, resolves, resolved.

Git vs GitHub



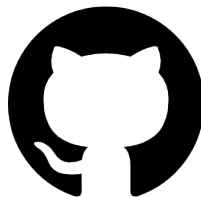
Git:

- ferramenta
- controle de versão
- local



GitHub / GitLab:

- hospedagem de repositórios
- colaboração
- CI/CD



Script vs Notebook



Script

- executável
- reprodutível
- melhor para produção
- múltiplos arquivos
- funciona bem com git

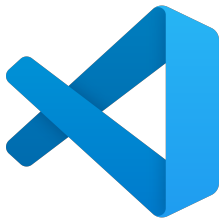


Notebook

- exploratório
- visual
- documentação interativa
- melhor para protótipos
- não funciona bem com git



IDEs



Visual Studio Code



Neovim



Windsurf



PyCharm



R Studio



Positron





- pip
- conda
- pipenv
- poetry
- uv

Listing 9: Criar projeto

```
1 uv init project
```

Listing 10: Adicionar dependência

```
1 uv add pandas
```

Arquivo central:

Listing 11: Config

```
1 {nvim, code} pyproject.toml
```

Obrigado!

Alguma dúvida?

Agora vamos para os exercícios!